

Answers for Week 9

1. CPU Matrix Multiplication

Before using the GPU, we will warm up with a CPU exercise to get used to Numba and JIT compiled code. In this exercise, we will explore implementations of matrix multiplication, checking the performance of each implementation and how it is related with how the CPU caches. Consider the following Python implementation of matrix multiplication:

```
1 import numpy as np
2
3 def matmul(A, B):
4     C = np.zeros((A.shape[0],B.shape[1]))
5     for i in range(A.shape[0]):
6         for j in range(B.shape[1]):
7             for k in range(A.shape[1]):
8                 C[i,j] += A[i,k] * B[k,j]
9     return C
```

1. Using the `@jit(nopython=True)` decorator, add Numba JIT compilation to the above function.

Answer: Add the decorator like this:

```
1 import numpy as np
2 from numba import jit
3
4 @jit(nopython=True)
5 def matmul_jit(A, B):
6     C = np.zeros((A.shape[0],B.shape[1]))
7     for i in range(A.shape[0]):
8         for j in range(B.shape[1]):
9             for k in range(A.shape[1]):
10                 C[i,j] += A[i,k] * B[k,j]
11     return C
```

2. Measure the wall time of the original and JIT compiled version for two 100×100 matrices. How much faster is the JIT compiled version? Remember to run the JIT compiled version once before timing so it can compile.
-

Answer: the following code will measure the run time, assuming the functions and matrices have been defined already:

```
1 from time import perf_counter as time
2
3 # Original
4 t = time()
5 for _ in range(10):
6     matmul(A, B)
7 print((time()-t) / 10 * 1000, 'ms')
8
9 # JIT compiled
10 matmul_jit(A, B) # Run once to compile
11 t = time()
12 for _ in range(1000): # Need more iterations for stable time
13     matmul_jit(A, B)
14 print((time()-t) / 1000 * 1000, 'ms')
```

The exact times will depend on your hardware. Using a Intel Xeon Gold 6142 CPU, I get a run time of 468.78 ms for the original function and 0.99 ms for the JIT compiled. A speed-up of over 470x!

-
3. **Autolab** Assuming A, B and C are stored row-wise, the innermost loop (over k) is not accessing B in a cache efficient manner. Make a new version of `matmul` where you re-order the loops such that access is cache efficient.

Answer: Autolab exercise - answer omitted

-
4. Measure the performance of your optimized version for $N \times N$ matrices, where N is at least 200. What do you observe?

Answer: The exact times will depend on your hardware. Using an Intel Xeon Gold 6142 CPU and 200 x 200 matrices I get 8.46 ms for the original loop order and 1.39 ms for the optimized loop order. Roughly a 6x speed-up by optimizing for cache efficiency!

-
5. (Optional) Measure the performance of your optimized version in MFLOP/s for $N \times N$ matrices where N goes from 20 to 2000. Plot the performance against the size of the matrix in kilobytes. Run the measurement as a batch job where you specify a CPU model so your results are repeatable. Mark the sizes of each cache level on your plot. What do you observe?

Hint: the matrix multiplication uses N^3 FLOPs.

Hint: recall how you measured performance from the exercises in week 3.

Answer: My job script was:

```
1 #!/bin/sh
2 ### General options
3 #BSUB -q hpc
4 #BSUB -J mm_cpu
5 #BSUB -n 1
6 #BSUB -R "span[hosts=1]"
7 #BSUB -R "rusage[mem=8GB]"
8 #BSUB -R "select[model == XeonGold6226R]"
9 #BSUB -W 00:20
10 #BSUB -o batch_output/matmul_cpu_%J.out
11 #BSUB -e batch_output/matmul_cpu_%J.err
12
13 # Initialize Python environment
14 source /dtu/projects/02613_2025/conda/conda_init.sh
15 conda activate 02613
16
17 lscpu
18
19 python -u timematmulcpu.py
```

My Python script for timing was:

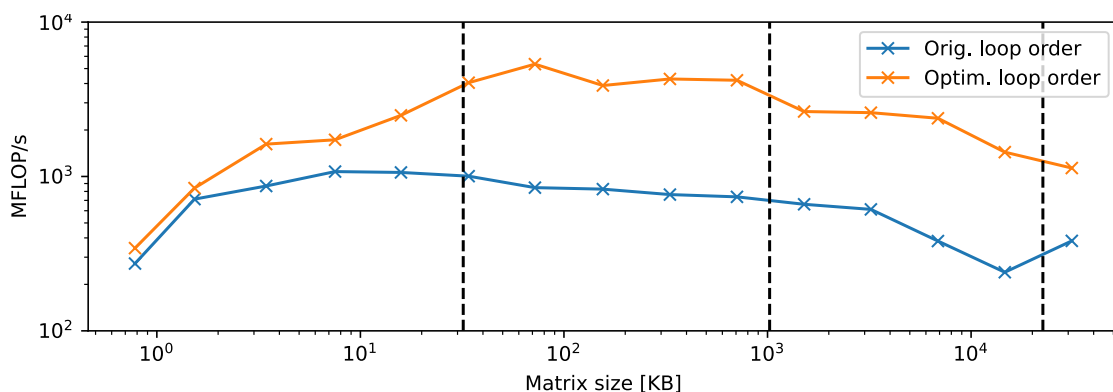
```
1 from time import perf_counter as time
2 import numpy as np
3 from numba import jit
4
5 @jit(nopython=True)
6 def matmul_jit(A,B):
7     C = np.empty((A.shape[0],B.shape[1]))
8     for i in range(A.shape[0]):
9         for j in range(B.shape[1]):
10             for k in range(A.shape[1]):
11                 C[i,j] += A[i,k]*B[k,j]
12     return C
13
14 @jit(nopython=True)
15 def matmul_jit_optim(A,B):
16     # Omitted
17
18 ns = np.logspace(1, 3 + np.log10(2), 15).astype(int)
19 rep = 10
20
21 A = np.random.rand(ns[0], ns[0])
22 B = np.random.rand(ns[0], ns[0])
```

```

23 # Run once to make sure it's jitted
24 dot1(A,B)
25 dot2(A,B)
26
27 t1, t2 = [], []
28 for n in ns:
29     A = np.random.rand(n, n)
30     B = np.random.rand(n, n)
31
32     t = time()
33     for i in range(rep):
34         dot1(A,B)
35     t1.append((time()-t) / rep)
36
37     t = time()
38     for j in range(rep):
39         dot2(A,B)
40     t2.append((time()-t) / rep)
41
42 print('ns = [' + ', '.join(map(str,ns)) + ']')
43 print('t1 = [' + ', '.join(map(str,t1)) + ']')
44 print('t2 = [' + ', '.join(map(str,t2)) + ']')

```

This resulted in the following performance plot:



The optimized loop order is significantly faster than the original one. Furthermore, we also see nice performance dips when the matrices no longer fit in each level of the cache.

2. CUDA Vector Addition

1. **Autolab** Make a CUDA kernel `add_kernel` that takes two vectors x and y as input and returns a new vector a where $a_i = x_i + y_i$, i.e., a is the element-wise sum of x and y .

Answer: Autolab exercise - answer omitted

2. Write a Python program that measures the run time of your kernel with two random vectors. Each input vector should be a NumPy array of length 1,000,000.
Hint: Run the kernel once before timing so it has been JIT compiled.
-

Answer:

```
1 from time import perf_counter as time
2 import numpy as np
3 from numba import cuda
4
5 @cuda.jit
6 def add_kernel(x, y, out):
7     # Omitted
8
9 n = 1_000_000
10 x = np.random.rand(n).astype(np.float32)
11 y = np.random.rand(n).astype(x.dtype)
12 out = np.empty_like(x)
13
14 threadsperblock = 256
15 blockspergrid = (n + (threadsperblock - 1)) // threadsperblock
16
17 add_kernel[blockspergrid, threadsperblock](x, y, out) # Ensure
    compiled
18
19 rep = 200
20 t = time()
21 for _ in range(rep):
22     add_kernel[blockspergrid, threadsperblock](x, y, out)
23 print((time()-t) / rep * 1000, 'ms')
```

3. **Autolab** Run your timing program as a batch job so results are repeatable. Use the queue c02613.
-

Answer: Autolab exercise - job script omitted.

From this, I get an average run time of 5.99 ms.

4. Repeat your time measurements where you ensure the arrays reside in pinned memory before passing them to your CUDA kernel. What do you observe and why?

Hint: To get pinned memory, use `numba.cuda.pinned_array`, [numba.cuda.pinned_array_like](#) or [numba.cuda.pinned](#).

Answer: I wrap the code from exercise 2.2 in the `cuda.pinned` context manager like this:

```
1 with cuda.pinned(A, B, C):
2     matmul_kernel[blockspersgrid, threadsperblock](A, B, C)
3
4     rep = 20
5     t = time()
6     for i in range(rep):
7         matmul_kernel[blockspersgrid, threadsperblock](A, B, C)
8     cuda.synchronize()
9     print((time() - t) / rep * 1000, 'ms')
```

This way, `x`, `y`, and `out` are pinned once. Using pinned memory, my run time decreases to 3.51 ms. This indicates that a significant part of the previous run time was simply transferring memory back and forth from the GPU.

Note: it is important that you do *not* repeatedly pin the memory just before each call to `add_kernel` in the timing. If you do that, you likely won't see any speed-up.

5. Repeat your time measurements where you ensure the arrays reside in GPU memory before passing them to your CUDA kernel. What do you observe and why? Based on your results, how much time is spent on data transfer vs. computation?

Answer: I change the last part of the Python code to

```
1 # Transfer to GPU
2 d_x = cuda.to_device(x)
3 d_y = cuda.to_device(y)
4 d_out = cuda.device_array(len(x))
5
6 add_kernel[blockspersgrid, threadsperblock](d_x, d_y, d_out) #
   Ensure compiled
7
8 rep = 200
9 t = time()
10 for _ in range(rep):
11     add_kernel[blockspersgrid, threadsperblock](d_x, d_y, d_out)
12 cuda.synchronize()
13 print((time()-t) / rep * 1000, 'ms')
```

I copy the `x` and `y` arrays to GPU memory before calling the kernel function with the `cuda.to_device` function. If needed, I could copy the result back to CPU memory using `out = d_out.copy_to_host()`. Now, the run time decreases to 0.042 ms.

Comparing with the run time for pinned CPU arrays, this means that about 1.2% was spent on actual computation and the remaining 98.28% were spent on memory transfer. *Moral:* with CUDA code, it is important to think how and when data moves between CPU and GPU!

Note: again, it is important that you only do the GPU transfer *\emph{once}* before you start timing.

3. CUDA Matrix Multiplication

1. Make a CUDA kernel `matmul_kernel(A, B, C)` that multiplies two matrices together. Let `A` and `B` be the input matrices and `C` be the output matrix you write to. You may assume the dimensions of `A` and `B` match so they can be multiplied. Structure your kernel such that you use a 2D compute grid where thread `(i, j)` computes the value of `C[i, j]`.
Hint: use `cuda.grid(2)` to get 2D grid position of the current thread.

Answer:

```
1 from numba import cuda, float32
2
3 @cuda.jit
4 def matmul_kernel(A, B, C):
5     i, j = cuda.grid(2)
6     if i < C.shape[0] and j < C.shape[1]:
7         tmp = float32(0.) # Avoids repeated mem. access
8         for k in range(A.shape[1]):
9             tmp += A[i, k] * B[k, j]
10        C[i, j] = tmp
```

2. Measure the run time of your kernel for `A` and `B` being 2048×2048 matrices and use 16×16 threads per block. Compare the run time with NumPy (e.g., by timing `A @ B`). What do you observe?

Answer: We can time the function using the following code snippet (assuming the function has already been defined and relevant modules imported):

```

1 d_A = cuda.to_device(A)
2 d_B = cuda.to_device(B)
3 d_C = cuda.device_array((A.shape[0], B.shape[1]))
4
5 matmul_kernel[blockspersgrid, threadsperblock](d_A, d_B, d_C)
6
7 rep = 20
8 t = time()
9 for i in range(rep):
10     matmul_kernel[blockspersgrid, threadsperblock](d_A, d_B, d_C)
11 cuda.synchronize()
12 print((time() - t) / rep * 1000, 'ms')

```

Notice that `threadsperblock` and `blockspersgrid` are now 2-tuples since we are defining a 2D grid.

Running this results in an average run time of 63 ms. Timing NumPy's matrix multiplication with `A @ B` results in a 101.83 ms. Our GPU implementation is already 1.6x faster.

-
3. Repeat your measurements where the matrices reside pinned and device memory. What fraction of time is spent on memory transfers to and from the GPU with and without pinned memory?
-

Answer: As before, I wrap the last lines in the `cuda.pinned` context manager:

```

1 with cuda.pinned(A, B, C):
2     matmul_kernel[blockspersgrid, threadsperblock](A, B, C)
3
4     rep = 20
5     t = time()
6     for i in range(rep):
7         matmul_kernel[blockspersgrid, threadsperblock](A, B, C)
8     cuda.synchronize()
9     print((time() - t) / rep * 1000, 'ms')

```

With this, the run time goes down to 50.61 ms. Now 2x faster than NumPy.

Manually placing the matrices in GPU memory before timing:

```

1 d_A = cuda.to_device(A)
2 d_B = cuda.to_device(B)
3 d_C = cuda.device_array((A.shape[0], B.shape[1]))
4
5 matmul_kernel[blockspersgrid, threadsperblock](d_A, d_B, d_C)
6
7 rep = 20
8 t = time()

```

```

9  for i in range(rep):
10     matmul_kernel[blockspgrid, threadspblock](d_A, d_B, d_C)
11  cuda.synchronize()
12  print((time() - t) / rep * 1000, 'ms')

```

Now, the run time further goes down to 42.06 ms, i.e., 2.4x faster than NumPy. Comparing to the pinned memory version, we can estimate that around 83% of the time is spend on actual computation and the remaining 17% on memory transfer. For the un-pinned version, only 67% of the run time is spent on computation. However, this is much better than the vector addition kernel, which makes sense as we now do significantly more work per thread.

-
4. Repeat your measurements using 256 x 1 and 1 x 256 threads per block. Let the matrices be in GPU memory before passing them to the kernel. What do you observe and why? Run your measurements as a batch job so results are repeatable.

Answer: First, notice that $16 \cdot 16 = 256$ so the amount of threads per block is always the same. Changing the grid to 256 x 1 threads per block increases the run time to 435.59 ms. To understand why, consider the for loop over k in our `matmul_kernel` function:

```

1  for k in range(A.shape[1]):
2     tmp += A[i, k] * B[k, j]

```

For the 256 x 1 configuration, each block will read along a column of A. This will incur a lot of cache misses resulting in the slow runtime. Conversely, for the 1 x 256 configuration, each block is reading along a row of B which is very cache efficient. As a result, the run time also decreases to 29.23 ms (compared to 42.06 ms for 16 x 16 thread blocks).

Moral of the story: we must, as always, ensure that our data access plays well with the cache. With CUDA, this also means that we must consider the layout of our compute grid, *in addition* to our code.
